

HealthRadar

Simulation de propagation de maladies en
milieu urbain

Membres du groupe ING 1 GI1-C :

TRAN TU THIEN Thémis

COLLIN Gweltaz

BEN GHORBEL Khayem

ABALE Marc-Antoine

GHOMMAM Mahdi

SAAD Mohamed

Tuteur : J. Mercadal

Sommaire :

Présentation du sujet :	3
Diagrammes UML & Use Case :	4
Diagramme UML :	4
Diagramme Use Case :	5
Différentes fonctionnalités :	6
Fonctionnalités de l'UI	6
Fonctionnalités dans la simulation	6
Règles de l'automate cellulaire	7
Dynamique de groupe:	8
Création du groupe :	8
Réunion de tuteurs :	8
Contribution individuelle du groupe :	9
Responsabilité par personnes :	9
Utilisation de l'IA :	10
Bases scientifiques & résultats :	11
Modèle de référence	11
Résistance individuelle — hétérogénéité de la population	12
Décroissance de la transmission aérienne avec la distance :	12
Les zones urbaines et le multiplicateur de transmission	13
Vaccination :	13
Port du masque — double efficacité	14
Contagion pré-symptomatique (état EXPOSED)	14
Problèmes rencontrés et solutions :	15
Mise en route et organisation :	15
Mauvaise architecture :	15
Différence d'apprentissage d'outils :	15
Gestion du temps :	16
Conclusion :	16
Références :	18

Présentation du sujet :

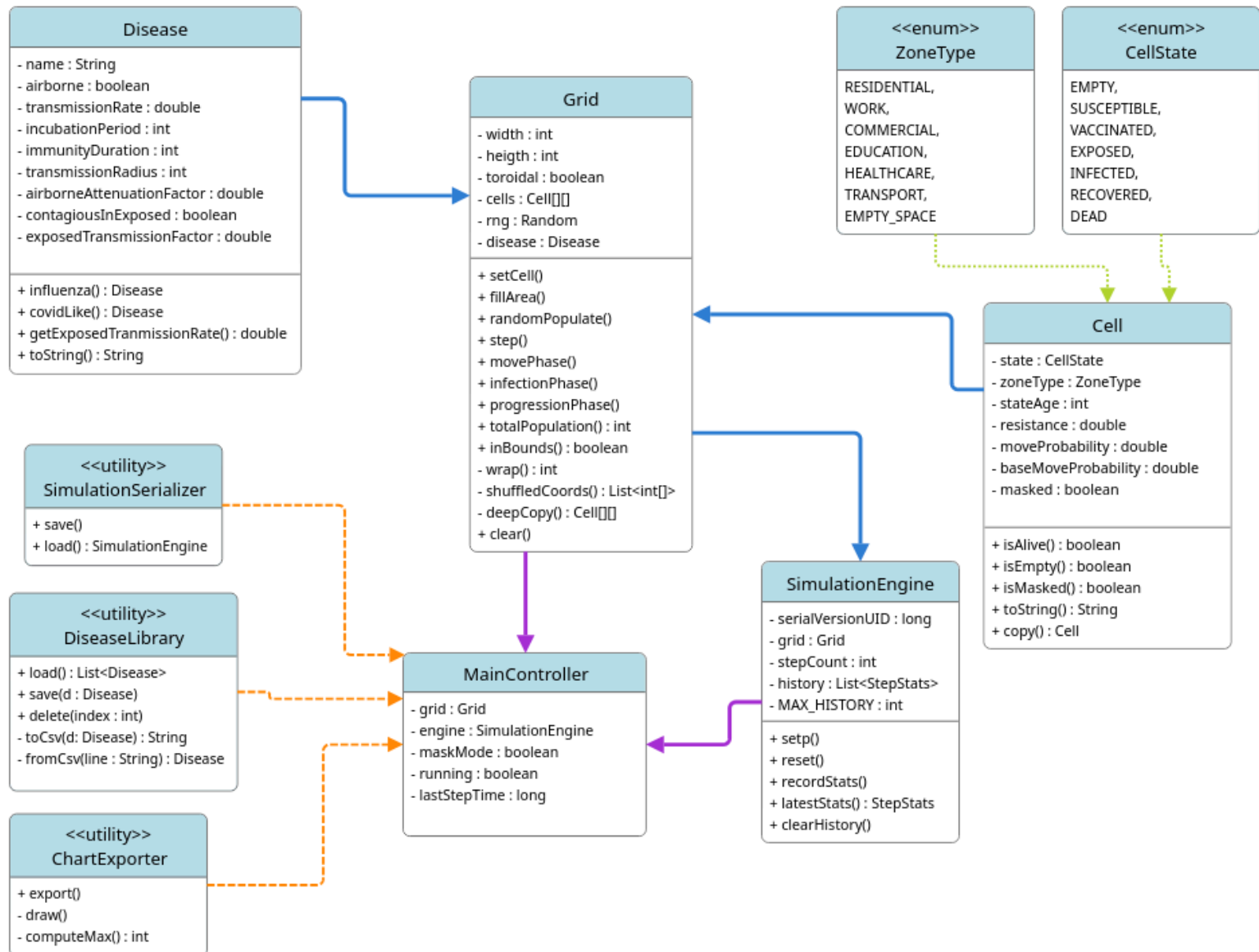
Notre équipe a choisi de réaliser un projet s'inscrivant dans la thématique n°2 : « Simulation de cellules dans un plan 2D ». L'objectif est de développer une application graphique en Java permettant de simuler le comportement de cellules évoluant sur une grille 2D, en fonction du temps, de l'environnement et des actions de l'utilisateur. Ce sujet nous a paru être le plus abordable parmi ceux proposés.

Notre groupe a donc décidé d'appliquer cette modélisation cellulaire à un enjeu de santé publique : la propagation de maladies en milieu urbain. Ce choix s'est imposé car il était le plus cohérent avec nos idées initiales. De plus, l'expérience de la Covid-19, qui a marqué chacun d'entre nous, nous a fait prendre conscience de l'importance de comprendre la dynamique de propagation des virus et de la nécessité d'informer le grand public le mieux possible pour lui permettre de se protéger.

HealthRadar est un automate cellulaire 2D dont l'objectif est de simuler la propagation de maladies infectieuses au sein d'une population évoluant dans une région urbaine. Chaque cellule de la grille représente soit une personne, soit un emplacement vide. Nous avons créé automate cellulaire 2D car, après plusieurs difficultés rencontrées lors de notre première approche, nous avons, grâce à notre tuteur, compris que c'était le modèle qui pouvait répondre le mieux aux exigences du cahier des charges. En effet, contrairement au modèle envisagé en premier lieu, celui-ci est beaucoup plus simple, stable et globalement plus performant, tout en restant cohérent avec notre problématique de départ et réaliste, car il intègre de nombreux paramètres concrets de la réalité d'une épidémie que ce soit au niveau du modèle épidémiologique de la transmission des différentes maladies.

Diagrammes UML & Use Case :

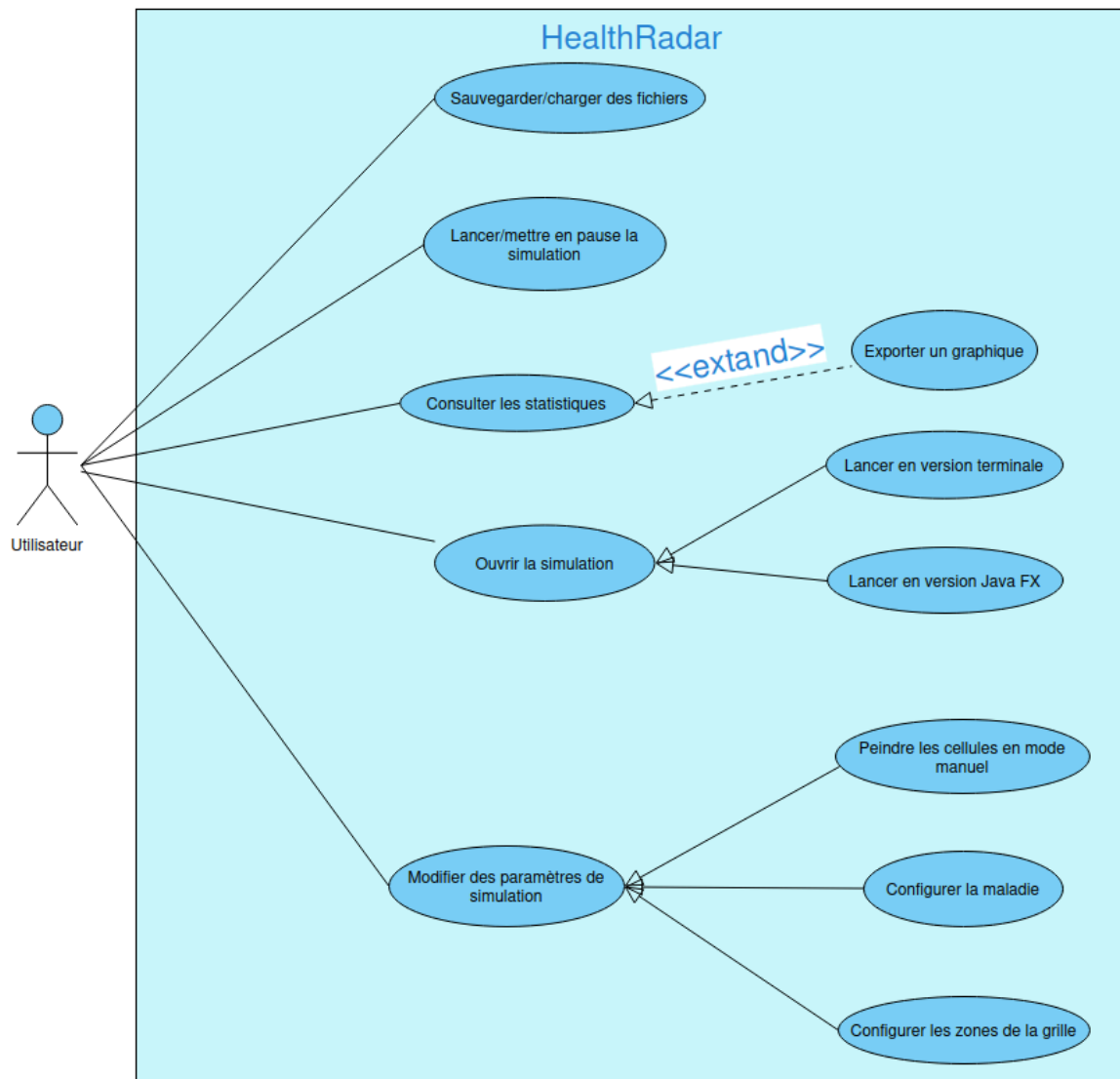
Diagramme UML :



Le noyau central correspond aux classes : Cell, Grid, et Disease avec les énumérations Zone Type et CellState.

La simulation tourne avec SimulationEngine et l'affichage que ce soit en JavaFX ou sur le terminal (CLI) est faite par MainController.

Diagramme Use Case :



L'utilisateur peut lancer la simulation que ce soit sur la version terminale (CLI). Il peut influencer sur la simulation en changeant des paramètres de maladie, de la grille ou utiliser le mode manuel. Il peut sauvegarder ou charger des fichiers de la grille et il peut consulter les statistiques tout en exportant un graphique en .png.

Différentes fonctionnalités :

Fonctionnalités de l'UI

L'interface utilisateur, qu'elle soit sur JavaFX ou sur le terminal a plusieurs fonctionnalités pour changer les paramètres de la simulation :

On peut modéliser différentes maladies, changer le taux de transmission, le temps d'incubation, d'infection, et le type de maladie (aérien/contact). On peut voir comment différents types de maladies meurent ou prospèrent dans le même milieu. De plus, on a créé des présets (influenza et COVID), afin de pouvoir commencer sans avoir à créer une toute nouvelle maladie.

On peut observer et avoir une image des statistiques de la simulation au cours du temps, ce qui nous permet d'étudier sur graphique les différents paramètres. On peut également sauvegarder et charger des simulations pour partir du même point de départ.

On a une fonctionnalité inspect, qui permet de voir les spécificités de chaque case, afin de voir les détails qui ne sont pas forcément affichés.

Il y a un menu settings pour les paramètres plus précis, comme la taille de l'affichage, le temps de la simulation. On peut aussi mettre en pause ou avancer jour par jour notre simulation. Ils nous permettent de mieux étudier comment évolue la population au fur et à mesure du temps.

Avec le mode manuel, on peut modifier les cases individuellement que ce soit l'état, le port du masque, ou la zone. Cela nous permet de créer des situations différentes et de voir l'influence de différents facteurs sur la propagation de maladie.

Fonctionnalités dans la simulation

Pour ce qui est de la simulation en elle-même, nous avons plusieurs aspects :

L'état vaccin et la fonctionnalité de masque ajoute une immunité et réduit la probabilité de transmission et d'infection. On peut modifier le pourcentage d'efficacité du vaccin, et celui du masque que ce soit l'utilisateur ou les personnes autour.

Le mouvement des personnes a été modélisé par une probabilité propre à chaque “personne”, car le mouvement peut varier. On donne une valeur aléatoire, et on utilise un curseur pour pouvoir modifier la probabilité de mouvement de la population entière. On fait cela pour modéliser les confinements, ou au contraire un mouvement accru.

Nous avons modélisé plusieurs types de maladies : les maladies de contact et les maladies aériennes. La maladie de contact utilise un voisinage de Moore (les 8 cases autour), alors que la maladie aérienne a un rayon de transmission autour d’elle. Elle nous permet de modéliser des maladies comme l’influenza ou des maladies de type COVID.

Pour l’aspect urbain, on a modélisé des zones qui représente, des zones résidentielles, des transports en commun, des bureaux de travail, des hôpitaux, des et des écoles. Pour chaque zone, on a un taux de transmission plus ou moins basé sur des études existantes [10].

Enfin, nous avons, avec notre simulation réussi à modéliser des phénomènes tels que les vagues de maladies et l’immunité collective. On arrive donc à reproduire avec les différentes maladies et l’espacement des personnes, des phénomènes réels.

Règles de l’automate cellulaire

Le changement d’état est guidé par des règles. Notre automate cellulaire est probabiliste, donc le départ d’une même base peut différencier d’une itération à l’autre. Un état “suseptible” ou “vacciné” devient “exposé” si la probabilité d’infection est atteinte. Au bout d’un certain nombre de jours, un état “exposé” devient “infecté”. A partir de cet état, à chaque tour est calculé la probabilité de mort (état “mort”). Au bout d’un certain temps, “l’infecté” devient automatiquement guéri (état “rétabli”). Les états “vaccinés” sont ceux mis manuellement. On ne peut pas passer d’un état quelconque à l’état vacciné sans l’intervention de l’utilisateur. L’état “mort” est un état final : il ne bouge plus et ne peut changer d’état.

Dynamique de groupe:

Création du groupe :

Notre groupe a été formé en début de semestre. Presque aucun des membres ne se connaissent. La moitié d'entre nous venaient de CPGE, et l'autre moitié venait du cursus PreIng. Nous n'étions donc pas dans les mêmes classes au semestre précédent. Avec les matières Ethiques, Anglais et Prototypage et tests, nous avons pu avancer sur notre projet commun et nous apprendre à nous connaître.

De plus, un de notre membre est arrivé plus tard. Marc-Antoine n'avait pas réussi à trouver de groupe dans la classe et donc, nous nous sommes retrouvé à être 6 dans le même groupe. Avec 6 personnes, il était difficile de s'organiser et de se mettre d'accord sur les choix du projet

Lors de nos présentations orales précédentes, nous étions assez efficace. Néanmoins avec les cours électifs, l'organisation du groupe était difficile pour le projet de Java, car personne n'a pris le lead avant 2 semaines du rendu. Certains essayaient de travailler de leurs côtés, mais aucune version claire et unanime n'était élu. Nos cours électifs étant différents par personne, nous ne nous retrouvions pas en physique autant qu'avant. Nous avons eu quelques réunions en présentiel, mais nous avons privilégié la communication en ligne et par conséquent, nous avons eu du mal à nous motiver entre nous.

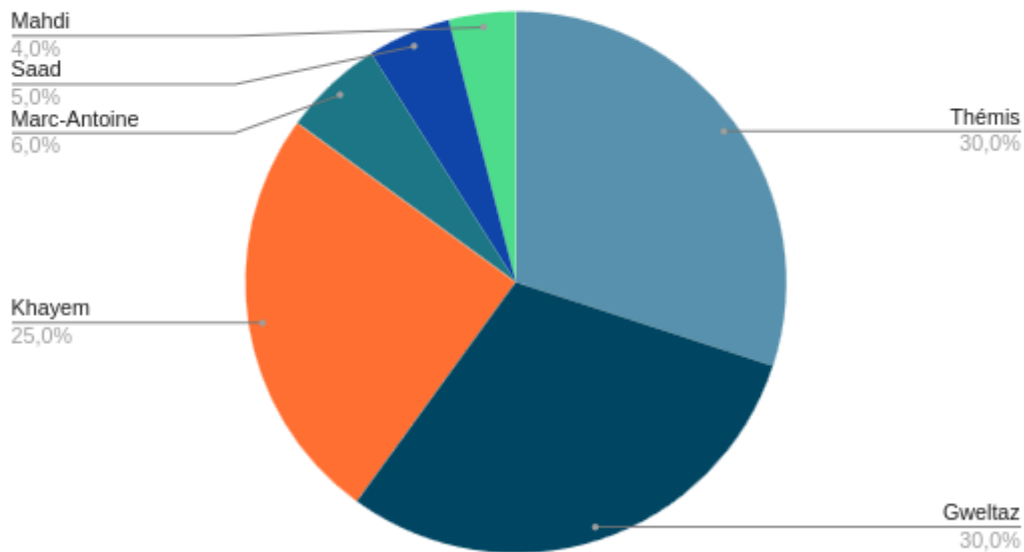
Réunion de tuteurs :

Les réunions avec notre tuteur M. Mercadal nous a à la fois aidé et démotivé. En effet, à notre première réunion, nous nous sommes rendu compte que le projet en Java était très différent que celui que nous avions prévu précédemment (lié aux autres matières). Nous avons donc eu l'impression qu'il fallait recommencer de 0. Lors de notre seconde réunion, nous avons repensé notre projet sous forme de personnes qui se déplacent dans des cases. Sauf que ce n'était pas exactement la forme d'un automate cellulaire. Donc bien que les réunions nous ont aidé à revenir sur "le droit chemin", elles nous ont fait nous rendre compte que beaucoup de travail a été fait pour rien, et nous ont beaucoup démotivé pour le travail à venir. Pendant la troisième et dernière réunion, nous

avons présenté une version qui répondait mieux aux attendus, montrant que les réunions précédentes étaient nécessaires.

Contribution individuelle du groupe :

Répartition du travail



Responsabilité par personnes :

Thémis : conception, fichiers de tests, corrections de bug, ajout de l'aspect urbain, fonctionnalité masques & vaccins, fonctionnalité du mouvement

Gweltaz : prototype du projet JavaFX, mode terminal, sauvegardes et fichier de sauvegarde, fonctionnalité masques & vaccins

Khayem : Premiers développements, UI JavaFX, conception, version Windows (Maven)

Saad : Transmission aérien proportionnelle

Mahdi : sliders masque et vaccin

Marc-Antoine : Commentaires, mouvement des populations

Utilisation de l'IA :

L'utilisation de l'IA nous a été fortement conseillée pour ce projet. En un mois, faire l'application JavaFX, le mode terminal, le système de sauvegarde, tout le modèle épidémiologique et la documentation, c'est beaucoup. On a donc utilisé l'IA, mais avec modération, en se mettant d'accord pour que cela reste un outil et pas une solution miracle.

D'abord pour coder certaines fonctionnalités, il fallait clairement définir ce qu'on voulait à l'avance. Le concept devait rester humain, tandis que le code lui-même pouvait être écrit par l'IA. Par exemple, la phase d'infection dans Grid.java, l'atténuation aérienne basée sur le modèle de Wells-Riley, les multiplicateurs de zones urbaines, la double efficacité du masque ou la sauvegarde binaire des fichiers .hrs. La méthode était toujours la même : on partait d'un principe scientifique trouvé dans nos lectures, on demandait une première version à l'IA, puis on la relisait, on la modifiait si besoin, et on l'intégrait au projet.

Ensuite pour la documentation, qui est sûrement l'endroit où l'IA nous a le plus aidés. La Javadoc sur les fichiers java ont été en grande partie rédigés avec son aide. Sachant que le concept a été expliqué à l'IA et relu

On avait quand même une crainte au début : finir avec du code qu'on ne comprendrait pas, ce qui aurait été un problème à la soutenance. Pour éviter ça, on a décidé de ne jamais intégrer une partie sans l'avoir relue et comprise. Et de toujours justifier les formules scientifiques par une source publiée, au lieu de croire l'IA sur parole. Au final, l'IA nous a permis d'aller plus loin et plus vite que ce qu'on aurait pu faire seuls dans le temps imparti.

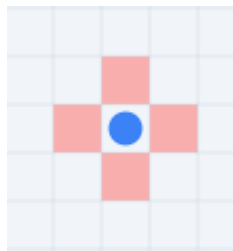
Bases scientifiques & résultats :

Cette partie justifie chaque formule mathématique présente dans le code de HealthRadar en la comparant aux modèles épidémiologiques publiés dans la littérature scientifique. Pour chaque mécanisme, nous présentons d'abord le modèle de référence, puis la formule implémentée dans le code Java, et enfin les éventuelles simplifications ou extensions apportées.

HealthRadar implémente un modèle épidémiologique de type SVEIRD (Susceptible, Vaccinated, Exposed, Infected, Recovered, Dead) dans un automate cellulaire 2D enrichi de zones urbaines et d'un modèle de décroissance de la transmission aérienne avec la distance. Chaque cellule possède donc différent état représenté par différentes couleurs :



À chaque tick, les cellules vivantes ont une probabilité de se déplacer vers une des 4 cases orthogonales adjacentes (haut, bas, gauche, droite).



Dans un automate cellulaire mis à jour en séquence, les cellules traitées en premier influencent celles traitées ensuite dans le même tick un biais dit d'aliasing temporel [1]. White et al. [3] identifient ce problème comme une source d'artefacts dans les CA épidémiologiques.

Pour palier à ce problème nous avons donc apporté la solution de la copie profonde + snapshot. Ce double mécanisme garantit que toutes les cellules "voient" exactement le même état du monde au début du tick, conformément à la définition formelle des AC synchrones [1, 2].

Modèle de référence

Dans un automate cellulaire SEIR, la probabilité qu'une cellule SUSCEPTIBLE soit infectée lors d'un contact avec une cellule INFECTED est modélisée par le paramètre β traduit en probabilité individuelle par tick [4, 3] :

$$P(S \rightarrow E) = \beta_{\text{discret}}$$

avec $\beta_{\text{discret}} \approx \beta_{\text{continu}} / (\text{nombre moyen de voisins})$

Cette probabilité de base constitue le point de départ. Dans HealthRadar, elle est ensuite modulée par l'ensemble des facteurs propres à chaque interaction, ce qui donne la formule complète implémentée dans `Grid.infectionPhase()` :

$$\begin{aligned}
P(S \rightarrow E) = & \beta \\
& / (d \times \alpha) \quad [\text{atténuation aérienne, uniquement si airborne}] \\
& \times \text{zoneMultiplier} \quad [\text{multiplicateur de zone de la source}] \\
& \times (1 - \text{maskOut}) \quad [\text{si la source porte un masque}] \\
& \times (1 - \theta) \quad [\text{si la cible est vaccinée}] \\
& \times (1 - \text{maskIn}) \quad [\text{si la cible porte un masque}] \\
& \times (1 - \text{resistance}) \quad [\text{résistance individuelle de la cible}]
\end{aligned}$$

Résistance individuelle — hétérogénéité de la population

Les modèles homogènes supposent que tous les individus ont la même susceptibilité. Or, les études épidémiologiques montrent une forte variabilité individuelle [6]. HealthRadar intègre cette hétérogénéité via un facteur de résistance individuelle dans Cell.java : `effectiveInfectionProbability()`

$P(\text{infection}) = \text{baseRate} \times (1 - \text{resistance})$

avec $\text{resistance} \sim N(\mu=0.20, \sigma=0.10) \rightarrow \text{this.resistance} = \text{Math.max}(0, \text{Math.min}(0.6, \text{rng.nextGaussian()} * 0.1 + 0.2))$, tronquée à $[0.0, 0.60]$

Cette formulation est cohérente avec les modèles hétérogènes de Lloyd-Smith et al. [5]

Décroissance de la transmission aérienne avec la distance :

La transmission par voie aérienne (aérosols, gouttelettes) ne se fait pas de façon uniforme dans un rayon fixe. Les études physiques et épidémiologiques montrent systématiquement une décroissance de la concentration virale et donc du risque d'infection avec la distance à la source.

La formule implémentée dans HealthRadar est dérivée directement du modèle de Wells-Riley [7], Riley et al. [8], qui est le modèle de référence pour la transmission aérienne en espace confiné. Ce modèle prédit que la concentration en particules infectieuses dans une pièce décroît proportionnellement à $1/d$, où d est la distance à la source.

En partant de ce principe, la probabilité d'infection à une distance d s'écrit :

$$P(d) = \beta / d$$

HealthRadar étend cette formule avec un paramètre α (*airborneAttenuationFactor*) qui permet d'ajuster la vitesse de décroissance selon le contexte physique nous permettant de simuler des lieux bien ventilés, super-confinés, etc :

$$P(d) = \beta / (d \times \alpha)$$

```
// Grid.java — infectionPhase() if (disease.isAirborne()) { double distance =
Math.sqrt(distanceSquared); // distance euclidienne double attenuation = distance *
disease.getAirborneAttenuationFactor(); if (attenuation > 0) baseRate /= attenuation; // baseRate =  $\beta /$ 
( $d \times \alpha$ ) }
```

Les zones urbaines et le multiplicateur de transmission

L'approche par matrices de contact est le standard pour modéliser l'hétérogénéité spatiale du risque épidémique. Dans ces études, le taux de contact moyen varie selon le lieu : [9]

- À domicile : contacts longs (8-12h) mais peu nombreux (2-4 personnes).
- Au travail : contacts moyens (4-8h) avec 5-20 personnes.
- Dans les transports : contacts courts mais densité très élevée (jusqu'à 8 personnes/m² dans un métro aux heures de pointe).

Zhang et al. [10] ont montré que réduire les contacts au travail et dans les transports avait un impact 3x plus fort par rapport aux contacts à domicile.

```
// ZoneType.java
RESIDENTIAL(1.0), // référence
WORK (1.4), // +40%
COMMERCIAL (1.2), // +20%
EDUCATION (1.5), // +50%
HEALTHCARE (1.8), // +80%
TRANSPORT (2.2), // +120%
EMPTY_SPACE(1.0) // neutre
```

```
// Grid.infectionPhase() application du multiplicateur
double zoneMultiplier = grid[r][c].getZoneType().getTransmissionMultiplier(); spreadRate[r][c] =
disease.getTransmissionRate() * zoneMultiplier;
```

Le multiplicateur s'applique à la source (le propagateur), pas à la cible. Cela modélise correctement le fait que la zone détermine la densité et la durée des contacts auxquels un individu est soumis dans ce lieu.

Vaccination :

Dans les modèles SEIR-V, la vaccination est modélisée par un paramètre d'efficacité θ qui réduit directement la probabilité qu'un individu vacciné soit infecté. La formulation standard, utilisée notamment par Sharma et al. et Abuin et al. est : [11, 12]

$$P_{\text{vacciné}} = P_{\text{base}} \times (1 - \theta)$$

Dans le code

```
// Grid.infectionPhase()  
// Vaccine inward protection  
if (tst == CellState.VACCINATED)  
    baseRate *= (1.0 - disease.getVaccineEfficacy());
```

Port du masque — double efficacité

7.1 Deux effets physiquement distincts Eikenberry et al.[13] sont les premiers à séparer explicitement dans un modèle SEIR les deux effets du masque :

- Inward : le masque filtre les particules inhalées par son porteur. Protège le porteur sain.
- Outward : le masque capture les particules expirées par son porteur. Protège l'entourage.

Implémentation dans Grid.java

```
// Outward : réduction du taux de propagation de la SOURCE  
    if (spreadRate[r][c] > 0 && grid[r][c].isMasked()) spreadRate[r][c] *=  
(1.0-disease.getMaskOutwardEfficacy()); // Inward : réduction de la probabilité d'infection de la  
CIBLE  
if (target.isMasked())  
    baseRate *= (1.0 - disease.getMaskInwardEfficacy());
```

Contagion pré-symptomatique (état EXPOSED)

He et al. [14] ont mesuré les charges virales de 77 patients COVID-19 et estiment que 44% des transmissions ont lieu avant l'apparition des symptômes, avec un pic de contagiosité autour de J-1 avant les symptômes. Dans les modèles SEIR classiques, l'état E (Exposed) est supposé non contagieux. Des extensions SEIR-pre-symptomatic (Liu et al., 2020 ; Ferretti et al., 2020 [15, 16]) ajoutent un taux de transmission réduit ϵ pour les exposés.

Dans notre code (Disease.java) le facteur 0.40 pour COVID-Like signifie qu'un individu en incubation est 40% aussi contagieux qu'un malade déclaré. Ce chiffre est cohérent avec les estimations de Ferretti et al. [15] qui évaluent la contribution pré-symptomatique à 37-48% des transmissions.

Problèmes rencontrés et solutions :

Mise en route et organisation :

La phase de démarrage a constitué notre première difficulté majeure. Le projet Java demandé différait énormément du travail que nous avons mené dans les matières Créativité & Innovation et Éthique, où nous avons conceptualisé notre simulation autour d'un modèle Ville->Case->Personne->Maladies (concept_initial.txt). Ce concept a été au final refusé par notre tuteur. Après cela, aucun membre du groupe ne savait par où commencer concrètement le travail, et personne n'a pris le lead pendant les premières semaines. Afin de débloquer, une petite partie du groupe a fini par poser une première base de code (prototype JavaFX, meilleure version du modèle) afin de donner un point de départ commun sur lequel les autres pouvaient se greffer.

Mauvaise architecture :

Le deuxième problème, identifié lors de notre première réunion avec notre tuteur M. Mercadal, était bien plus structurel : notre conception initiale, fondée sur des personnes se déplaçant librement entre des cases, ne correspondait pas à la définition d'un automate cellulaire. Il a donc fallu repenser entièrement le projet. Nous sommes passés d'un modèle centré sur des entités mobiles à un modèle où la cellule de la grille est l'unité atomique de la simulation : chaque case contient une personne, avec son état, ou un emplacement vide, et l'évolution se fait par règles locales appliquées en parallèle à chaque tick (movement phase, infection phase, progression phase). Cette refonte a impliqué de jeter une partie significative du travail conceptuel précédent, ce qui a été démotivant, mais elle a permis d'aboutir à un modèle stable, performant et conforme à la définition d'un automate cellulaire 2D.

Différence d'apprentissage d'outils :

La différence d'apprentissage de technique entre les membres venant des CPGE et ceux du cursus PréIng a constitué un frein. Les outils du projet : organisation d'un projet, et gestion d'un dépôt github ; n'avaient pas été abordés en classe préparatoire, ce qui a créé une asymétrie de productivité entre les

membres. Pour régler ce problème nous n'avons pas trouvé de solution "parfaite", en nous appuyant sur l'entraide interne.

Gestion du temps :

La gestion du temps a également été un problème. La consigne explicite encourageant l'usage de l'IA traduisait, le fait qu'un livrable de cette envergure n'était pas réalisable en un mois sans assistance. Nous avons donc intégré l'IA dans notre développement, ce qui a immédiatement fait apparaître un cinquième et dernier problème : le risque de livrer du code que nous ne maîtriserions pas. La solution adoptée a été une règle simple : toute portion de code générée devait être relue, comprise et au besoin réécrite avant intégration, afin que chaque partie du code soit en mesure d'être expliquée par un membre du groupe pendant la soutenance.

Conclusion :

Pour conclure, le projet n'a pas été parfait, mais on en sort avec une application fonctionnelle, qui répond aux attentes. HealthRadar simule la propagation d'une maladie en zone urbaine modélisée par une grille 2D, avec une interface JavaFX et un mode terminal, plusieurs types de zones urbaines, des mécanismes de protection (masques, vaccination), et un modèle épidémiologique basé sur des références scientifiques. L'utilisateur peut configurer sa simulation, suivre les courbes en temps réel et sauvegarder son travail.

Nous pensons que le résultat aurait pu être de meilleure qualité avec une meilleure organisation au départ du projet. Le lien entre ce projet dans différentes matières était également assez flou. Nous avions une idée propre de notre projet dans les matières d'humanité, pourtant lors du passage au code fonctionnel, nous avons eu l'impression de refaire un tout autre projet. Le fait que l'intégration de l'IA nous a paru obligatoire si l'on était à l'aise avec Java nous a paru démotivant. Malgré tout, ce projet a été une expérience pour coopérer avec des personnes que l'on ne connaissait pas.

Références :

- [1] Wolfram, S. (2002). *A New Kind of Science*. Wolfram Media. <https://www.wolframscience.com/nks/>
- [2] Von Neumann, J. (1966). *Theory of Self-Reproducing Automata*. University of Illinois Press.
- [3] White, S.H. et al. (2007). Modeling epidemics using cellular automata. *Applied Mathematics and Computation*, 186(1), 193–202. <https://doi.org/10.1016/j.amc.2006.06.126>
- [4] Sirakoulis, G.C. et al. (2000). A cellular automaton model for the effects of population movement and vaccination on epidemic propagation. *Ecological Modelling*, 133(3), 209–223. [https://doi.org/10.1016/S0304-3800\(00\)00294-5](https://doi.org/10.1016/S0304-3800(00)00294-5)
- [5] Lloyd-Smith, J.O. et al. (2005). Superspreading and the effect of individual variation on disease emergence. *Nature*, 438, 355–359. <https://doi.org/10.1038/nature04153>
- [6] Goel, R.R. et al. (2021). Distinct antibody and memory B cell responses in SARS-CoV-2 naïve and recovered individuals. *Science Immunology*, 6(58). <https://doi.org/10.1126/sciimmunol.abi6950>
- [7] Wells, W.F. (1955). *Airborne Contagion and Air Hygiene*. Harvard University Press.
- [8] Riley, R.L. et al. (1978). Aerial dissemination of pulmonary tuberculosis. *American Journal of Epidemiology*, 107(5), 369–375. <https://doi.org/10.1093/oxfordjournals.aje.a112560>
- [9] Mossong, J. et al. (2008). Social contacts and mixing patterns relevant to the spread of infectious diseases. *PLoS Medicine*, 5(3), e74. <https://doi.org/10.1371/journal.pmed.0050074>
- [10] Zhang, J. et al. (2020). Changes in contact patterns shape the dynamics of the COVID-19 outbreak in China. *Science*, 368(6498), 1481–1486. <https://doi.org/10.1126/science.abb8001>
- [11] Sharma, P. et al. (2024). Optimal vaccination strategies in the control of an infectious disease: a SEIRV model. *arXiv:2411.04910*. <https://arxiv.org/abs/2411.04910>
- [12] Abuin, P. et al. (2021). Characterization of SARS-CoV-2 dynamics in the host. *Annual Reviews in Control*, 50, 457–468. <https://arxiv.org/abs/1103.4993>
- [13] Eikenberry, S.E. et al. (2020). To mask or not to mask. *Infectious Disease Modelling*, 5, 293–308. <https://doi.org/10.1016/j.idm.2020.04.001>
- [14] He, X. et al. (2020). Temporal dynamics in viral shedding and transmissibility of COVID-19. *Nature Medicine*, 26, 672–675. <https://doi.org/10.1038/s41591-020-0869-5>
- [15] Ferretti, L. et al. (2020). Quantifying SARS-CoV-2 transmission suggests epidemic control with digital contact tracing. *Science*, 368(6491). <https://doi.org/10.1126/science.abb6936>
- [16] Liu, Y. et al. (2020). Secondary attack rate and superspreading events for SARS-CoV-2. *The Lancet*, 395(10227), e47. [https://doi.org/10.1016/S0140-6736\(20\)30462-1](https://doi.org/10.1016/S0140-6736(20)30462-1)